

网页多模态 Agent（界面理解与自动操作）实验报告

摘要

网页浏览器是人类获取信息和完成在线任务的主要入口。近年来，大语言模型（Large Language Model, LLM）与视觉语言模型（Vision-Language Model, VLM）的发展，使得“根据自然语言指令自主操作网页”的网页 Agent 成为一个重要研究方向。本项目围绕“网页多模态 Agent（界面理解 + 自动操作）”任务，设计并实现了一个基于本地开源多模态模型、结构化网页观察信息和 Playwright 浏览器自动化工具的网页任务执行系统。系统接收自然语言任务，在真实浏览器环境中循环执行页面观察、动作规划、浏览器操作、成功判断和轨迹记录，支持点击、输入、回车、滚动、等待和结束等基础动作。

系统实现上，本项目采用“VLM/LLM Planner + DOM 解析 + 页面截图 + Playwright 执行器”的模块化架构。页面观察模块从浏览器中提取 URL、标题、DOM 可见文本、可点击元素列表、元素位置和截图路径；Prompt 构造模块将任务目标、成功条件、页面状态和历史动作组织为结构化输入；模型规划模块调用本地 Qwen2.5-VL-7B-Instruct 生成严格 JSON 格式的单步动作；动作执行模块通过 Playwright 在真实网页中完成操作；成功判断模块根据 URL、标题、正文文本或元素文本等条件判定任务是否完成；日志模块保存每一步事件、截图和最终 summary，便于复现和错误分析。除命令行批量实验外，项目还补充实现了一个 Gradio Web UI 演示界面，用于现场展示自然语言任务输入、浏览器自动操作、动作日志和截图序列。

实验方面，最终批量实验在云服务器上使用 Qwen2.5-VL-7B-Instruct 完成，任务覆盖 Wikipedia 与 GitHub 两类真实网站，共 6 个搜索类任务场景。主实验采用 screenshot+DOM 输入模式，每个任务运行 10 次，共 60 次；对照实验采用 DOM-only 输入模式，共 31 次。结果显示，在本项目构造的稳定搜索类任务集合中，screenshot+DOM 模式 60/60 成功，平均步数为 3.87；DOM-only 模式 31/31 成功，平均步数为 4.19。该结果说明，在任务相对规范、网页结构稳定且成功条件清晰的情况下，本地开源 7B 级多模态模型结合结构化 DOM 信息可以稳定完成一批真实网页自动化任务。与此同时，早期调试中较小模型（如 Qwen2.5-VL-3B）经常出现页面识别不稳定、动作重复、未提交搜索、JSON 输出不规范等问题，因此最终定量实验采用 Qwen2.5-VL-7B-Instruct，并将小模型表现作为系统局限和错误分析的一部分讨论。

本项目的主要贡献包括：第一，完成了一个可本地运行、可复现、可记录的网页多模态 Agent 工程闭环；第二，设计了面向浏览器动作的结构化 JSON 动作空间和输出解析兜底机制；第三，在真实网站上完成了 60 次以上批量实验，并提供成功率、平均步数、任务级结果与截图轨迹；第四，实现了交互式 Web UI 演示模块，使系统从命令行评测扩展到可视化展示形态。实验也表明，当前系统仍主要适用于搜索、打开结果、页面跳转等中低复杂度任务，对于复杂表单、多轮网页交互、强视觉定位、动态验证码或页面结构频繁变化的任务仍存在明显挑战。

关键词： 网页 Agent；多模态大模型；Qwen2.5-VL；Playwright；浏览器自动化；DOM；视觉语言模型；任务规划

\newpage

1. 引言

1.1 研究背景

随着互联网应用日益复杂，许多日常任务都需要用户在网页中完成多步操作，例如搜索资料、打开文档、填写表单、筛选商品、登录系统、提交请求和整理页面信息。传统浏览器自动化通常依赖人工编写脚本，例如使用 Selenium 或 Playwright 指定 CSS selector、XPath 或固定坐标进行点击和输入。这类方法在任务固定、页面结构稳定时效果很好，但面对自然语言任务、网页结构变化和开放域网站时，往往需要大量手工规则，泛化能力有限。

大语言模型和多模态模型为网页自动化提供了新的技术路径。LLM 具备自然语言理解、任务分解和动作规划能力；VLM 能够理解截图中的视觉布局、文本和图标；浏览器自动化工具则能够真实执行点击、输入、滚动等操作。如果将这些能力组合起来，就可以构建一种网页 Agent：用户只需给出自然语言目标，Agent 通过观察网页状态，规划下一步动作，并调用工具执行，最终完成任务。

本项目正是围绕这一方向展开。课程题目要求设计一个网页多模态 Agent，基于网页截图、结构信息和自然语言指令，生成一系列浏览器操作（点击、输入、滚动等）来完成具体任务，并能在浏览器自动化环境中演示。根据题目要求，系统不仅要实现操作流程，还需要提供定量实验、可视化轨迹、错误分析和最终展示。

1.2 问题定义

给定一个网页任务，形式化表示为：

$$T = (u_0, I, C)$$

其中， u_0 表示起始 URL， I 表示自然语言任务指令， C 表示成功条件。Agent 在每一步获得当前网页观察：

$$O_t = (url_t, title_t, dom_t, elements_t, screenshot_t, history_t)$$

系统需要根据任务 T 和观察 O_t 生成一个浏览器动作：

```
A_t ∈ {click, click_xy, type, press, scroll, wait, finish}
```

执行动作后网页进入新状态。循环持续到成功条件满足、模型输出 finish、达到最大步数或执行失败。项目关注的问题是：本地部署的开源多模态模型能否结合 DOM、截图和浏览器工具，在真实网站上稳定完成一组网页任务，并生成可复现的操作轨迹。

1.3 设计目标

本项目的设计目标包括以下几个方面：

1. **可执行性。** Agent 输出必须能够被浏览器自动化工具执行，不能停留在自然语言描述层面。
2. **可复现性。** 每次运行需要保存任务配置、模型动作、执行结果、页面截图和最终 summary。
3. **可评测性。** 系统需要支持批量运行任务，并统计成功率、平均步数和失败原因。
4. **可扩展性。** 系统应当将浏览器环境、Planner、模型客户端、任务加载和成功判断解耦，方便替换模型或增加任务。
5. **可展示性。** 除命令行批量实验外，系统应支持可视化演示，展示 Agent 如何逐步操作浏览器。

1.4 本文结构

本文后续结构如下：第 2 节介绍相关工作，包括浏览器自动化、语言模型 Agent、网页环境 benchmark 和多模态网页 Agent；第 3 节介绍系统方法，包括总体架构、页面观察、动作空间、Prompt 构造、模型调用、JSON 解析、成功判断和 Web UI；第 4 节介绍实验设置、任务集合、模型配置、结果统计和错误分析；第 5 节讨论系统局限与未来改进方向；第 6 节给出结论；最后列出参考文献。

\newpage

2. 相关工作

2.1 浏览器自动化工具

浏览器自动化长期以来是软件测试、网页爬取和流程自动化的重要技术。Selenium WebDriver 提供了跨浏览器的自动化接口，可以通过元素选择器、键盘鼠标事件和页面等待机制驱动浏览器。Playwright 是较新的浏览器自动化框架，支持 Chromium、Firefox 和 WebKit，提供更现代的自动等待、上下文隔离、截图、网络控制和定位器机制。本项目选择 Playwright 作为执行工具，原因是其 Python API 简洁，截图、元素定位和 headless/headful 运行模式都适合网页 Agent 实验。

传统自动化脚本通常由程序员预先写好操作逻辑，例如“找到搜索框，输入关键词，按 Enter”。这种方法可控性强，但难以根据自然语言目标自动适配新任务。网页 Agent 的目标不是完全取代自动化工具，而是让模型负责高层动作规划，让 Playwright 负责底层可靠执行。

2.2 LLM Agent 与 ReAct 框架

LLM Agent 的核心思想是让语言模型不仅生成文本，还能在推理过程中调用外部工具。ReAct 框架提出将 Reasoning 和 Acting 结合起来，让模型交替进行思考和行动，从而完成问答、交互式决策等任务。网页 Agent 可以被视为 ReAct 在浏览器环境中的应用：模型根据页面状态进行推理，输出动作，工具执行后返回新观察，模型继续规划下一步。

本项目借鉴了 ReAct 的循环式思想，但在工程实现上做了约束：模型每次只允许输出一个 JSON 动作，而不是自由格式的思考文本和工具调用。这种设计牺牲了一部分表达自由度，但显著提高了动作可解析性和执行安全性。

2.3 网页交互 Benchmark

MiniWoB++ 是较早用于网页交互任务的 benchmark，包含大量小型网页环境和明确任务目标，适合训练和评估模型的页面操作能力。WebArena 则构建了更接近真实互联网应用的环境，涵盖购物、论坛、GitLab、地图等多类网站，任务更复杂、更接近真实用户需求。VisualWebArena 进一步强调视觉信息在网页任务中的作用，要求 Agent 利用截图和页面视觉布局完成任务。

这些工作说明，网页 Agent 的评估需要同时考虑页面理解、任务规划、动作执行和成功判断。与大型 benchmark 相比，本项目规模较小，主要面向课程项目实现与验证；但系统仍保留了 task 配置、成功条件、批量运行和 trace 记录等评测框架，为后续扩展到更复杂 benchmark 提供基础。

2.4 多模态网页 Agent

随着 GPT-4V、Qwen-VL、LLaVA、InternVL 等多模态模型的发展，研究者开始探索让模型直接理解网页截图并执行操作。SeeAct 等工作研究了将视觉语言模型用于网页动作预测的问题，证明多模态模型可以从截图和网页上下文中推断可交互元素。Open-source Agent 项目如 OpenClaw、Nanobot 等也展示了将 LLM 与浏览器工具结合的工程可行性。

本项目采用开源 Qwen2.5-VL-7B-Instruct 作为本地多模态模型。与只依赖截图的方式不同，本项目同时提供 DOM 文本、可点击元素列表、元素 bbox 和截图路径，使模型既能利用结构化网页信息，也能在需要时结合视觉信息。实验中还设置 DOM-only 对照，用于观察截图信息对任务步数和稳定性的影响。

\newpage

3. 方法

3.1 系统总体架构

本项目采用模块化架构，整体流程为：

```
TASK -> OBSERVE -> PLAN -> ACT -> CHECK_SUCCESS -> LOG
```

对应的主要模块如下：

```
web_agent/  
  browser/  
    env.py          # Playwright 浏览器环境封装  
    actions.py      # 点击、输入、滚动等动作执行  
    observation.py   # DOM、可点击元素和截图提取  
  agent/  
    runner.py        # Agent 主循环  
    llm_planner.py   # LLM/VLM Planner  
    rule_based_planner.py  
  models/  
    hf_client.py     # Hugging Face 本地模型客户端  
    prompt_builder.py # Prompt 构造  
    json_parser.py   # JSON 动作解析和兜底  
  eval/  
    task_loader.py   # YAML 任务加载  
    success_checker.py # 成功条件判断
```

AgentRunner 是系统的核心调度器。每个任务开始时，Runner 创建 trace 目录并启动浏览器，然后打开任务起始 URL。每一轮循环中，系统首先调用 `get_observation()` 获取页面状态；然后调用 `check_success()` 判断是否已经完成任务；如果尚未完成，则调用 Planner 根据任务、观察和历史动作生成下一步动作；随后由浏览器环境执行动作，并记录结果。当成功条件满足、模型输出 finish、连续失败或达到最大步数时，系统结束任务并写入 `summary.json`。

3.2 页面观察表示

网页状态具有多模态和半结构化特点。单纯使用截图会带来视觉定位和 OCR 难题，单纯使用 DOM 又可能遗漏视觉布局和页面显著性。因此，本项目构造了混合观察表示：

1. **URL 与标题。** 用于判断页面跳转是否完成，也为模型提供当前页面语义。
2. **DOM 可见文本。** 从页面 body 中提取可见文本，并根据最大字符数截断。
3. **可点击元素列表。** 提取 `a`、`button`、`input`、`textarea`、`select`、`role=button`、`aria-label`、`onclick` 等元素。
4. **元素属性。** 包括 tag、text、aria_label、placeholder、href、title、role、input_type、selector 等。

- 5. **元素位置。** 记录 bbox 中的 x、y、width、height，必要时可用于坐标点击。
- 6. **页面截图。** 保存当前页面截图路径，供 VLM 输入或后续可视化分析使用。
- 7. **历史动作。** 提供最近若干步动作和结果，减少重复点击或重复输入。

可点击元素被编号为 `element_id`。Planner 优先输出基于 `element_id` 的 click 动作，而不是直接输出坐标。这样可以减少屏幕分辨率、缩放比例和页面滚动带来的坐标误差。

3.3 动作空间设计

为了保证模型输出可执行，本项目定义了有限动作空间：

动作	参数	含义
click	element_id	点击提取出的网页元素
click_xy	x, y	坐标点击，仅在没有合适 element_id 时使用
type	text	向当前聚焦输入框输入文本
press	key	按键，例如 Enter
scroll	direction	向上、下、左、右滚动
wait	seconds	等待页面加载或动画完成
finish	answer	任务完成，输出最终回答

每次模型只能输出一个动作。这一约束有三点好处：第一，方便执行与调试；第二，可以在每一步后重新观察页面状态，适应动态网页变化；第三，便于记录细粒度轨迹和错误原因。

3.4 Prompt 构造

Prompt 构造模块将任务和页面观察组织为严格模板。模板中包括任务 ID、网站类型、自然语言指令、成功条件、当前 URL、标题、截图路径、可点击元素列表、DOM 文本和最近动作历史。Prompt 明确要求模型：

- 1. 只输出一个 JSON 对象；
- 2. 不输出 Markdown；
- 3. 不输出解释性文本；
- 4. 只使用允许动作；
- 5. 优先使用 element_id；
- 6. 如果需要搜索，先点击搜索框，再输入查询词，再按 Enter；
- 7. 如果成功条件满足，则输出 finish；

8. 不要重复执行同一个动作超过两次。

这种 Prompt 设计将自由形式网页操作问题转化为结构化动作预测问题。相比让模型直接输出自然语言步骤，结构化 JSON 更容易被程序解析，也更适合批量评测。

3.5 模型客户端与本地推理

项目提供 MockModelClient 和 Hugging Face 本地模型客户端两类模型接口。Mock 模型用于无 GPU 环境下验证工程闭环，例如检查任务加载、浏览器启动、动作执行和成功判断是否正常。真实实验使用本地 Qwen2.5-VL-7B-Instruct 模型，通过 Hugging Face Transformers 进行推理。

最终云服务器批量实验采用如下模型与参数：

```
Model: Qwen2.5-VL-7B-Instruct
Model path: /data1/xiangkun/MODELS/Qwen2.5-VL-7B-Instruct
max_steps = 12
max_new_tokens = 128
max_dom_chars = 3000
max_elements = 20
post_action_sleep = 1
```

早期曾尝试较小模型，例如 Qwen2.5-VL-3B。调试过程中发现，小模型在网页动作规划中更容易出现以下问题：无法稳定定位搜索框、输出动作字段不规范、重复点击或重复输入、输入搜索词后没有提交、对成功条件理解不充分等。项目保留了一个 3B 失败样例，位于 `traces/3B失败样例/`：该样例使用 Qwen2.5-VL-3B-Instruct 执行 `wiki_search_web_agent_results` 任务，浏览器成功打开 Wikipedia 首页，但模型连续多轮输出 `{"action": "type", "text": "Web agent"}`，没有先点击搜索框，也没有按 Enter 提交搜索，最终达到 `max_steps=12` 后失败。由于 3B 实验没有形成与 7B 主实验相同规模的批量统计，本文不虚构其总体成功率，而是将该 trace 作为模型规模影响和错误分析中的真实失败案例。最终报告中的定量成功率仍全部来自 Qwen2.5-VL-7B-Instruct 的批量实验。

3.6 JSON 解析与安全兜底

真实模型并不总是严格遵守输出格式，可能输出 Markdown 代码块、解释性文字、字段别名或嵌套 JSON。为此，系统实现了 JSON 解析和规范化模块，支持以下情况：

1. 纯 JSON 输出；
2. Markdown fenced code block 中的 JSON；
3. 前后带自然语言解释的 JSON；
4. `type`、`action_type` 等字段别名；
5. `target_id`、`target` 到 `element_id` 的别名转换；
6. `value`、`content` 到 `text` 的别名转换；

7. 嵌套 action 对象扁平化。

如果无法解析出合法动作，系统不会直接崩溃，而是兜底为：

```
{"thought": "Invalid model output", "action": "wait", "seconds": 1}
```

对于 click 动作，系统还会检查 element_id 是否存在于当前可点击元素列表中。如果模型输出不存在的元素编号，则同样转为安全等待动作。这种机制减少了模型偶发输出错误对整个任务运行的破坏。

3.7 成功条件判断

每个任务在 YAML 中定义成功条件。系统支持多种成功判断方式：

类型	示例	含义
url_contains	/wiki/Web_agent	当前 URL 包含目标字符串
text_contains	Selenium	页面文本包含目标关键词
title_contains	Playwright	页面标题包含目标关键词
element_text_contains	指定元素文本	页面元素文本包含目标内容
manual_check	人工检查	用于无法自动判断的任务

成功判断在每轮规划前执行。如果当前页面已经满足条件，Runner 会在下一步动作前结束任务，避免模型继续进行不必要操作。若模型输出 finish，则系统也会再次检查成功条件，防止模型主观认为完成但页面实际未满足目标。

3.8 轨迹记录与可视化

系统为每次运行创建独立 trace 目录，保存：

```
events.jsonl
summary.json
screenshots/
```

其中，events.jsonl 记录每一步观察、动作、执行结果和错误信息；summary.json 记录任务配置、最终 URL、成功状态、总步数、失败原因和历史动作；screenshots/ 保存操作前后页面截图。这些文件既用于实验统计，也用于展示和错误分析。

最终结果还整理为 CSV 与图表，包括：


```
results/qwen25vl_all_runs.csv
results/qwen25vl_summary_by_mode.csv
results/qwen25vl_summary_by_task_and_mode.csv
results/figures/success_rate_by_mode.png
results/figures/avg_steps_by_mode.png
results/figures/per_task_success_rate_screenshot_dom.png
results/figures/per_task_success_rate_dom_only.png
```

3.9 Web UI 演示模块

项目在命令行批量评测之外增加了交互式 Web UI。Web UI 基于 Gradio 实现，入口为 `webui/app.py`。用户可以在网页中输入自然语言任务，选择运行模式，并实时查看动作日志、最新截图和输出目录。

Web UI 支持两种模式：

1. **project_agent_cli**。调用原项目的 VLM/LLM Agent 入口，适合展示完整模型推理流程。
2. **browser_demo_fallback**。使用轻量 Playwright 规则演示器，适合本地或现场快速展示，保证浏览器动作可见。

需要强调的是，Web UI 的 fallback 模式主要用于可视化演示，不参与最终定量指标统计。最终报告中的成功率和平均步数来自云服务器上 Qwen2.5-VL-7B-Instruct 的批量实验。

\newpage

4. 实验

4.1 实验环境

最终批量实验在云服务器上完成，模型为 Qwen2.5-VL-7B-Instruct，模型路径为：

```
/data1/xiangkun/MODELS/Qwen2.5-VL-7B-Instruct
```

浏览器自动化使用 Playwright 和 Chromium。实验采用 headless 模式运行，以便在服务器上稳定批量执行。关键参数如下：

参数	数值
模型	Qwen2.5-VL-7B-Instruct
最大步数	12
最大生成 token	128
DOM 最大字符数	3000
可点击元素最大数量	20
动作后等待	1 秒
主实验输入	screenshot+DOM
对照实验输入	DOM-only

本地展示实验在 Windows 环境中完成，使用同一项目目录下的 `.conda` Python 环境运行 Web UI。Web UI 地址为 `http://127.0.0.1:7860`，演示模式使用 `browser_demo_fallback`，并关闭 `headless` 以展示真实浏览器窗口。

4.2 任务集合

最终批量实验覆盖 2 个真实网站、6 个搜索类任务。任务配置位于 `eval/tasks.yaml`。任务如下：

任务 ID	网站	任务描述	成功条件
wiki_search_web_agent	Wikipedia	搜索 Web agent 并打开相关页面	URL 包含目标页面
wiki_search_playwright	Wikipedia	搜索 Playwright 并打开结果页	URL 包含 Playwright
wiki_search_selenium	Wikipedia	搜索 Selenium software 并打开结果页	页面文本包含 Selenium
github_search_playwright	GitHub	搜索 playwright 仓库	URL 包含 search
github_search_web_agent	GitHub	搜索 web agent 仓库	URL 包含 search
github_search_qwen_vl	GitHub	搜索 qwen vl 仓库	URL 包含 search

这些任务满足课程要求：选择 1-2 个真实网站，每个网站设计至少 3 个任务场景，每个场景多次独立测试。任务类型以搜索和打开结果为主，复杂度适中，便于验证 Agent 基础闭环。

4.3 评价指标

本文使用以下指标：

1. 任务成功率。

```
success_rate = success_runs / total_runs
```

- 2. 平均步数。 任务完成前执行的平均动作步数，越低表示执行更高效。
- 3. 失败原因。 若任务失败，记录 max_steps、错误元素、搜索未提交、成功条件不匹配、环境错误等原因。
- 4. 可视化轨迹。 保存截图序列和动作日志，用于人工检查执行过程。

由于最终批量实验没有产生失败样本，失败原因统计为空。本文不会虚构失败数量，而是在错误分析部分结合早期调试和系统结构讨论潜在失败类型。

4.4 3B 小模型失败样例

除最终 7B 批量实验外，项目还保留了一个 Qwen2.5-VL-3B-Instruct 的失败样例，用于说明模型规模较小时网页 Agent 容易出现的典型问题。该样例不计入 7B 主实验成功率，而作为错误分析案例使用。

项目	内容
Trace 目录	traces/3B失败样例/
模型	Qwen2.5-VL-3B-Instruct
任务	wiki_search_web_agent_results
起始页面	https://www.wikipedia.org/
目标	搜索 Web agent 并使 URL 包含 Web_agent
结果	失败
失败原因	达到 max_steps=12
主要现象	连续多轮输出 type Web agent ，没有点击搜索框或按 Enter 提交

该失败样例表明，小模型不一定缺乏输出 JSON 的能力；它的问题更多体现在状态理解和动作序列控制上。即使 Prompt 中明确给出了“先点击搜索框、再输入、再按 Enter”的规则，3B 模型仍可能停留在局部动作上，不能根据历史动作切换到下一步。这一现象支持了本文最终选择 Qwen2.5-VL-7B-Instruct 作为批量实验模型的原因。

4.5 主实验结果：screenshot+DOM

主实验使用 screenshot+DOM 输入模式，在 6 个任务上每个任务运行 10 次，共 60 次。结果如下：

输入模式	运行次数	成功次数	失败次数	成功率	平均步数
screenshot+DOM	60	60	0	100%	3.87

任务级结果如下：

任务 ID	运行次数	成功次数	成功率	平均步数
github_search_playwright	10	10	100%	4.0
github_search_qwen_vl	10	10	100%	4.0
github_search_web_agent	10	10	100%	4.0
wiki_search_playwright	10	10	100%	4.0
wiki_search_selenium	10	10	100%	3.2
wiki_search_web_agent_results	10	10	100%	4.0

从结果看，Qwen2.5-VL-7B-Instruct 在本任务集合上表现稳定。大多数任务需要 4 步左右完成，典型流程为：打开起始页、点击搜索框、输入关键词、按 Enter 或点击搜索结果、满足成功条件。wiki_search_selenium 平均步数较低，为 3.2，说明该任务的成功条件较容易通过页面文本满足。

4.6 对照实验结果：DOM-only

对照实验关闭截图输入，仅使用 DOM 文本和可点击元素列表。由于对照实验运行次数为 31 次，不完全等同于主实验的 60 次，但仍可用于观察 DOM-only 模式的基本表现。

输入模式	运行次数	成功次数	失败次数	成功率	平均步数
DOM-only	31	31	0	100%	4.19

任务级结果如下：

任务 ID	运行次数	成功次数	成功率	平均步数
github_search_playwright	5	5	100%	5.0
github_search_qwen_vl	5	5	100%	5.0
github_search_web_agent	5	5	100%	4.0
wiki_search_playwright	5	5	100%	4.0
wiki_search_selenium	5	5	100%	3.2
wiki_search_web_agent	6	6	100%	4.0

DOM-only 也达到 100% 成功率，说明对于搜索类任务，结构化 DOM 和可点击元素列表已经提供了足够信息。但 DOM-only 平均步数为 4.19，高于 screenshot+DOM 的 3.87，尤其 GitHub 的 Playwright 和 qwen vl 搜索任务平均步数为 5.0。这表明截图信息可能有助于模型更快确认页面状态或选择更合适元素，但在当前任务规模下差异不大。

4.7 两种输入模式对比

输入模式	运行次数	成功率	平均步数
screenshot+DOM	60	100%	3.87
DOM-only	31	100%	4.19

两种模式均成功完成所有最终批量任务，说明系统的结构化网页观察对搜索任务非常关键。截图并没有显著改变成功率，但略微降低了平均步数。可能原因包括：

- 1. 搜索任务页面结构清晰，DOM 和可点击元素足以定位搜索框。
- 2. 成功条件多为 URL 或文本包含，容易自动判断。
- 3. 任务路径较短，不需要复杂视觉推理。
- 4. screenshot+DOM 为模型提供了页面布局补充，使其在某些页面上减少等待或重复动作。

因此，本文不能将 100% 成功率泛化到所有网页任务。更合理的解释是：在本项目构造的稳定搜索任务集合上，结构化 DOM + 本地 7B 多模态模型已经能够形成可靠闭环；截图信息在该任务集合中主要改善效率，而非决定成败。

4.8 可视化结果

项目生成了多张统计图，可用于报告排版和展示：


```
results/figures/success_rate_by_mode.png
results/figures/avg_steps_by_mode.png
results/figures/per_task_success_rate_screenshot_dom.png
results/figures/per_task_success_rate_dom_only.png
```

此外，典型成功案例保存在：

```
results/final/cases/dom_only_success/
results/final/cases/screenshot_dom_success/
```

每个案例目录包含 `events.jsonl`、`summary.json` 和 `screenshots/`。这些轨迹可以展示 Agent 每一步的页面截图、动作类型和执行结果。例如，一个典型 Wikipedia 搜索任务会依次经历打开首页、点击搜索输入框、输入关键词、按 Enter、跳转到目标页面并满足 URL 条件。

4.9 Web UI 演示实验

本地 Web UI 运行了 3 个自然语言任务，结果保存在：

```
results/webui_runs/
```

三次运行结果如下：

运行目录	任务	模式	状态	说明
run_20260603_160454	去 Wikipedia 搜索 WebAgent, 并打开相关页面总结第一段	browser_demo_fallback	SUCCESS	打开搜索结果并生成摘要
run_20260603_160523	在百度搜索 Qwen2.5-VL, 并打开第一个可信结果	browser_demo_fallback	SUCCESS	完成搜索并展示页面
run_20260603_160553	去 Playwright 官网搜索 locator 文档	browser_demo_fallback	FAILED / NEEDS HUMAN VERIFICATION	搜索页触发真人验证, 未能继续打开官网文档

其中，Playwright locator 任务没有完成到目标官网文档页，原因是搜索结果页面出现真人验证/反爬拦截，导致 Agent 无法继续点击第一个结果。这类失败不应被记为任务成功，而应归为外部环境或网页访问限制导致的执行中断。该现象也说明 Web UI fallback 模式虽然适合现场演示“自然语言到浏览器动作”的可视化过程，但其运行结果仍会受到搜索引擎验证、网络状态和网页反爬策略影响。因此报告将 Web UI 作为演示模块，而不是定量实验依据。

Web UI 的价值主要体现在：

- 1. 用户不需要输入复杂命令，可以直接在网页中输入自然语言任务。
- 2. 操作日志和截图实时展示，便于上台讲解 Agent 的感知—规划—执行过程。
- 3. 支持关闭 headless，浏览器窗口可直接录屏。
- 4. 与批量实验解耦，不影响正式结果统计。

4.10 错误分析

最终 Qwen2.5-VL-7B-Instruct 批量实验没有产生失败样本，因此本文不构造虚假的 7B 失败统计。但项目额外保留了一个 Qwen2.5-VL-3B-Instruct 的失败 trace，可作为小模型失败案例分析。该样例位于 `traces/3B失败样例/`，任务为 `wiki_search_web_agent_results`，失败原因为 `Reached max_steps=12`。从 `events.jsonl` 可以看到，模型在 Wikipedia 首页连续输出 `type Web agent` 动作，没有执行“点击搜索框—输入关键词—按 Enter”的完整序列，最终 URL 始终停留在 `https://www.wikipedia.org/`，成功条件“URL 包含 Web_agent”没有被满足。结合该失败样例、早期调试和 Web UI 演示，可以总结潜在失败类型：

1. **识别失败。** 模型无法从截图或 DOM 中定位正确搜索框，尤其当页面存在多个输入框、弹窗或广告区域时更明显。
2. **规划失败。** 模型重复输出同一动作，例如反复点击搜索框，或者输入关键词后没有按 Enter。
3. **执行失败。** Playwright 找不到元素、元素不可见、页面尚未加载完成或点击被遮挡。
4. **格式失败。** 模型输出不是严格 JSON，或者字段名不符合动作空间要求。
5. **成功判断失败。** 页面实际上已经接近目标，但 URL 或文本不满足预设 success condition。
6. **外部环境失败。** 网络波动、网站结构变化、GitHub/Wikipedia 页面加载失败、代理不稳定、搜索引擎真人验证或反爬拦截等。
7. **模型规模限制。** Qwen2.5-VL-3B 等小模型在动作规划和格式遵循上明显不如 7B 模型稳定。

3B 失败样例中最突出的现象是动作重复和搜索未提交：模型反复认为下一步仍应输入 `Web agent`，但没有根据历史动作切换到 `press Enter`，也没有通过 URL 变化判断任务尚未推进。这说明较小模型虽然能够生成合法 JSON 动作，但在状态跟踪和动作序列控制方面仍不稳定。为此，系统加入了 JSON parser、字段别名兼容、非法 `element_id` 检查、历史动作提示和连续失败停止机制。这些工程措施对于提升最终 7B 批量实验稳定性非常重要。

4.11 结果讨论

实验结果说明，网页 Agent 的成功不仅依赖模型能力，还依赖工程约束。有限动作空间、结构化可点击元素、严格 JSON 输出、成功条件检查和轨迹记录共同降低了任务难度，使 7B 级本地模型能够稳定完成搜索类任务。

同时，结果也提示当前评测任务仍偏简单。Wikipedia 与 GitHub 搜索任务通常路径较短，页面结构清晰，不涉及登录、复杂表单、动态弹窗、多条件筛选或跨页面状态记忆。因此，100% 成功率应理解为“系统在当前任务集上的稳定性验证”，而不是“网页 Agent 已经具备通用网页操作能力”。如果扩展到更复杂任务，预计会出现更多识别、规划和执行失败。

\newpage

5. 局限性与未来工作

5.1 任务复杂度有限

本项目任务主要集中在搜索和打开结果页面，属于网页操作中的基础能力。虽然这些任务能够验证观察、规划、执行和成功判断闭环，但不足以全面评估 Agent 在真实复杂网站中的能力。未来可以扩展到购物网站筛选、表单填写、文档检索、多标签页操作、登录后任务和长程信息整理。

5.2 最终失败样本不足

最终批量实验没有失败样本，这对展示稳定性有利，但不利于量化错误分布。早期小模型和调试阶段确实出现过较多失败，但由于没有统一保存为最终 CSV，本文只能进行定性分析。未来应当保留所有中间失败实验，并按模型、任务、失败类型进行系统标注。

5.3 小模型稳定性不足

Qwen2.5-VL-3B 等小模型在本任务中经常失败，说明网页 Agent 对模型的指令遵循、结构化输出和页面理解能力要求较高。未来可以尝试：

1. 为小模型设计更短、更明确的 Prompt；
2. 引入动作模板或规则约束；
3. 使用轨迹数据进行 LoRA 微调；
4. 将 VLM 仅用于元素识别，将 LLM 负责规划；
5. 使用更强的 JSON constrained decoding。

5.4 成功条件仍偏人工设计

当前 success condition 由任务配置人工指定，例如 URL 包含某个字符串。这种方法简单可靠，但无法处理开放式任务，例如“找到最相关的论文并总结贡献”。未来可以引入语义级成功判断，例如基于 LLM 的页面内容评估、任务答案检查或人工审核结合。

5.5 视觉信息利用仍不充分

虽然系统支持 screenshot+DOM，但当前任务中 DOM 已经足够强，视觉信息的增益较小。未来应设计更多依赖视觉布局的任务，例如识别图标按钮、处理无文本控件、根据页面区域点击、理解图表或图片内容，从而更充分评估多模态能力。

5.6 安全与鲁棒性问题

真实网页 Agent 可能执行高风险操作，例如提交表单、删除内容、购买商品或泄露隐私。本项目只在公开网页上执行低风险搜索任务。未来若扩展到真实生产环境，需要加入权限控制、动作确认、敏感信息过滤、沙盒浏览器环境和日志审计机制。

6. 结论

本项目实现了一个基于本地开源多模态模型的网页多模态 Agent 系统。系统通过 Playwright 控制真实浏览器，通过页面截图、DOM 文本和可点击元素列表构造网页观察，通过 Qwen2.5-VL-7B-Instruct 生成结构化 JSON 动作，并在每一步执行后进行成功判断和轨迹记录。项目完成了从任务配置、浏览器环境、模型规划、动作执行、结果保存到 Web UI 展示的完整工程闭环。

实验结果表明，在 Wikipedia 与 GitHub 的 6 个搜索类任务上，最终 Qwen2.5-VL-7B-Instruct 批量实验表现稳定。screenshot+DOM 主实验共 60 次运行全部成功，平均步数 3.87；DOM-only 对照实验 31 次全部成功，平均步数 4.19。结果说明，对于结构清晰、目标明确的搜索类网页任务，本地 7B 级多模态模型结合结构化 DOM 信息已经可以稳定完成自动化操作。截图信息在当前任务中主要带来轻微效率提升，而不是决定性成功率提升。

项目同时表明，网页 Agent 的可靠性高度依赖工程设计。严格动作空间、JSON 输出约束、元素编号、历史动作提示、成功条件检查和失败兜底机制是保证系统稳定运行的关键。早期小模型实验中出现的识别失败、规划失败和格式失败也说明，模型规模和指令遵循能力对网页 Agent 表现有显著影响。

最终，项目不仅完成了课程要求的系统实现和批量实验，还实现了交互式 Web UI 演示模块，可在本地浏览器中展示自然语言任务输入、浏览器自动操作、动作日志和截图序列。未来工作可进一步扩展任务复杂度、保留更多失败样本、强化视觉任务评测、引入语义成功判断，并探索面向网页动作轨迹的模型微调，从而提升 Agent 在真实开放网页环境中的泛化能力和鲁棒性。

\newpage

参考文献

- [1] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. ReAct: Synergizing Reasoning and Acting in Language Models. arXiv:2210.03629, 2022.
<https://arxiv.org/abs/2210.03629>
- [2] Zhou, S., Xu, F. F., Zhu, H., Zhou, X., Lo, R., Sridhar, A., Cheng, X., Bisk, Y., Fried, D., Alon, U., & Neubig, G. WebArena: A Realistic Web Environment for Building Autonomous Agents. arXiv:2307.13854, 2023. <https://arxiv.org/abs/2307.13854>
- [3] Koh, J. Y., Lo, R., Jang, L., Duvvur, V., Lim, M. C., Huang, P.-Y., Neubig, G., Zhou, S., Salakhutdinov, R., & Fried, D. VisualWebArena: Evaluating Multimodal Agents on Realistic Visual Web Tasks. arXiv:2401.13649, 2024. <https://arxiv.org/abs/2401.13649>
- [4] Shi, T., Karpathy, A., Fan, L., Hernandez, J., & Liang, P. World of Bits: An Open-Domain Platform for Web-Based Agents. Proceedings of ICML, 2017.
<https://proceedings.mlr.press/v70/shi17a.html>

- [5] Bai, S., Chen, K., Liu, X., Wang, J., Ge, W., Song, S., Dang, K., Wang, P., Wang, S., Tang, J., et al. Qwen2.5-VL Technical Report. arXiv:2502.13923, 2025. <https://arxiv.org/abs/2502.13923>
- [6] Bai, J., Bai, S., Yang, S., Wang, S., Tan, S., Wang, P., Lin, J., Zhou, C., & Zhou, J. Qwen-VL: A Versatile Vision-Language Model for Understanding, Localization, Text Reading, and Beyond. arXiv:2308.12966, 2023. <https://arxiv.org/abs/2308.12966>
- [7] Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E., Zhang, H., Gonzalez, J. E., & Stoica, I. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. NeurIPS, 2023. <https://arxiv.org/abs/2306.05685>
- [8] Playwright Documentation. Browser automation and end-to-end testing framework. <https://playwright.dev/>
- [9] Selenium Documentation. WebDriver browser automation documentation. <https://www.selenium.dev/documentation/webdriver/>
- [10] OpenAI. GPT-4V(ision) System Card. 2023. <https://openai.com/research/gpt-4v-system-card>
- [11] Deng, X., Gu, Y., Zheng, B., Chen, S., Stevens, S., Wang, B., Sun, H., & Su, Y. Mind2Web: Towards a Generalist Agent for the Web. NeurIPS, 2023. <https://arxiv.org/abs/2306.06070>
- [12] Kim, G., Baldi, P., & McAleer, S. Language Models can Solve Computer Tasks. arXiv:2303.17491, 2023. <https://arxiv.org/abs/2303.17491>
- [13] Zheng, B., Gou, B., Kil, J., Sun, H., & Su, Y. GPT-4V(ision) is a Generalist Web Agent, if Grounded. arXiv:2401.01614, 2024. <https://arxiv.org/abs/2401.01614>
- [14] SMART Lab Purdue. SMART-LLM Multi-Agent Task Planning. <https://github.com/SMARTlab-Purdue/SMART-LLM>
- [15] Gradio Documentation. Building machine learning web interfaces in Python. <https://www.gradio.app/docs>

\newpage

附录 A：复现命令

A.1 安装依赖

```
pip install -r requirements.txt
python -m playwright install chromium
```


Web UI 额外依赖:

```
pip install -r requirements_webui.txt
```

A.2 运行单个 Qwen2.5-VL 任务

```
export VLM_MODEL_PATH=/data1/xiangkun/MODELS/Qwen2.5-VL-7B-Instruct

CUDA_VISIBLE_DEVICES=0 python examples/demo_local_llm_planner.py \
  --task-id wiki_search_web_agent \
  --model "$VLM_MODEL_PATH" \
  --headless \
  --use-screenshot \
  --max-steps 12 \
  --max-new-tokens 128 \
  --max-dom-chars 3000 \
  --max-elements 20 \
  --post-action-sleep 1
```

A.3 查看结果

```
cat results/qwen25vl_summary_by_mode.csv
cat results/qwen25vl_summary_by_task_and_mode.csv
```

A.4 启动本地 Web UI

Windows PowerShell:

```
cd E:\上课\多模态\大作业\web_agent\web_agent_final_submission
& "E:\上课\多模态\大作业\web_agent\conda\python.exe" webui\app.py --server-name
127.0.0.1 --server-port 7860
```

打开:

```
http://127.0.0.1:7860
```